



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Casting and Aliases

Let's assume you are given a data file. You are told that data is dirty. So you must clean it up and save it as a Spark table. Once the data is clean, we will be using it for analysis. Right? That's a common requirement.

You want to start working on this requirement. So you asked for the following things:

1. Sample data file
2. File schema
3. Known data problems

You need a sample data file to test your code using the sample data and verify if your logic is working. And you got a directory location where some data files are there. You have 2TB of data. That's too big, and you cannot use such volumes for development and testing. But that's fine. You at least have data to start the work. Then you asked for file schema so you can read the given data, and you got the schema as shown below.

Requirement:

Data Cleaning and Preparation

What do you need:

1. Sample data file (2 TB data file)
2. File schema
3. Known data problems

Purchase date is in YYYY-MM-DD or DD-MMM-YYYY

id int	-> Transaction id
name string	-> Customer Name
dop string	-> Date of purchase
phone long	-> Customer Phone Number
amount string	-> Purchase Amount
discount string	-> Discount Percentage

And they give you only one known problem.

Some date of purchase values are in YYYY-MM-DD format, but we also have values in DD-MM-YYYY format.

Requirement:

Data Cleaning and Preparation

What do you need:

1. Sample data file (2 TB data file)
2. File schema
3. Known data problems

→ Purchase date is YYYY-MM-DD or DD-MM-YYYY

id int	-> Transaction id
name string	-> Customer Name
dop string	-> Date of purchase
phone long	-> Customer Phone Number
amount string	-> Purchase Amount
discount string	-> Discount Percentage

The first thing is to formulate the solution or make a list of things we want to do. I prepared the following list:

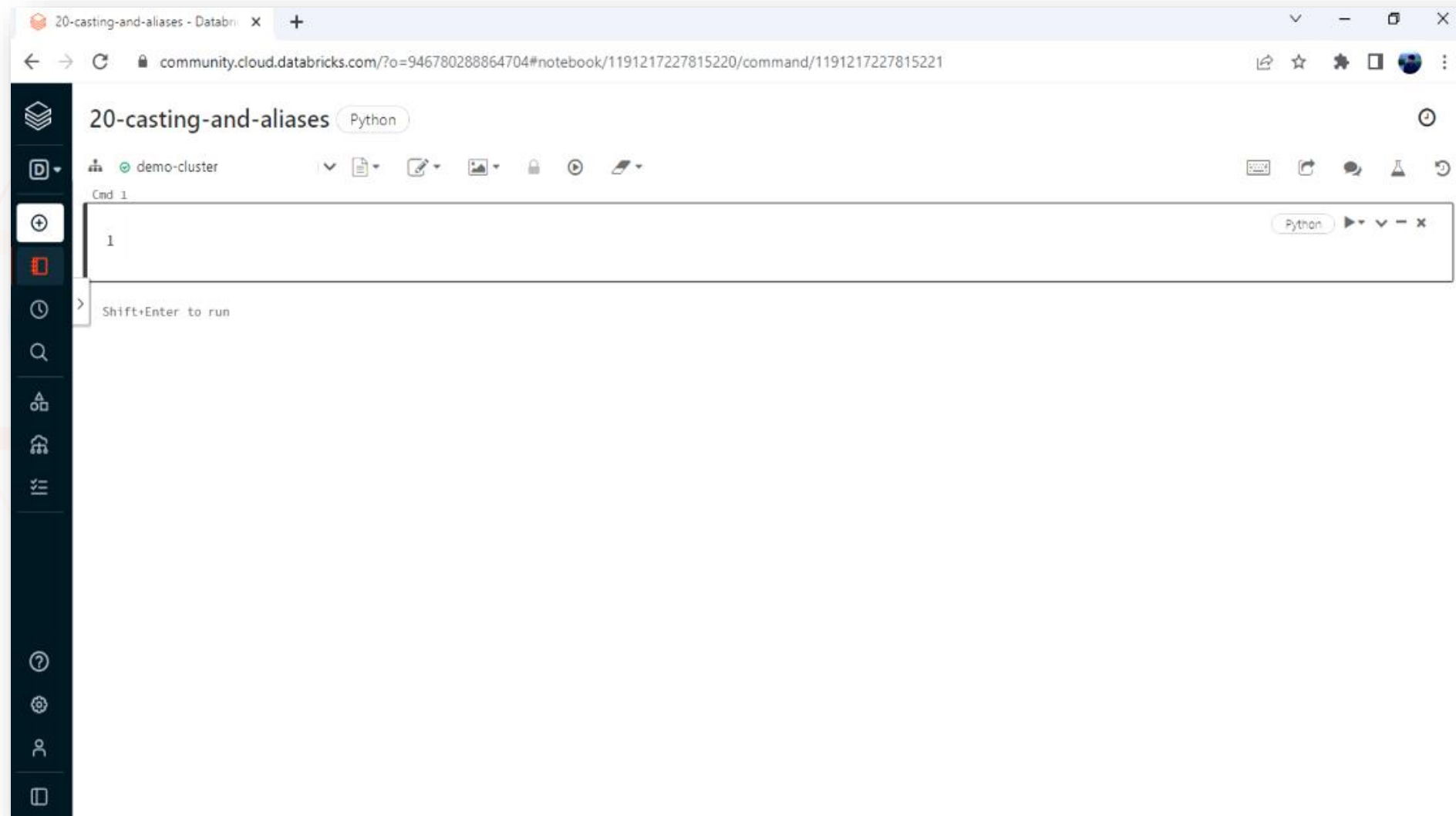
1. Create a small but good quality sample dataset
2. Analyse dataset to identify data problems
3. Ensure the sample includes problem records
4. Develop logic to correct and segregate
5. Save the correct records in a Spark table
6. Save the incorrect records to an error table

You need a small sample dataset for development and testing. You cannot work with a 2TB dataset for development and testing. That's too expensive and time-consuming also.

So the first step is to read the given data and create a sample Dataframe.

You can read the given data and take 10K samples from it. We have a known problem with different date formats. So you wanted to make sure that you are sampling data from YYYY-MM-DD and DD-MM-YYYY formats. So you can also go for a stratified sample for the date of purchase column.

Go to your Databricks workspace and create a new notebook. **(Reference : 20-casting-and-aliases)**



I want to create a sample Dataframe for the given requirement manually. So, I have defined a data list and a schema for my requirement.

20-casting-and-aliases

Python

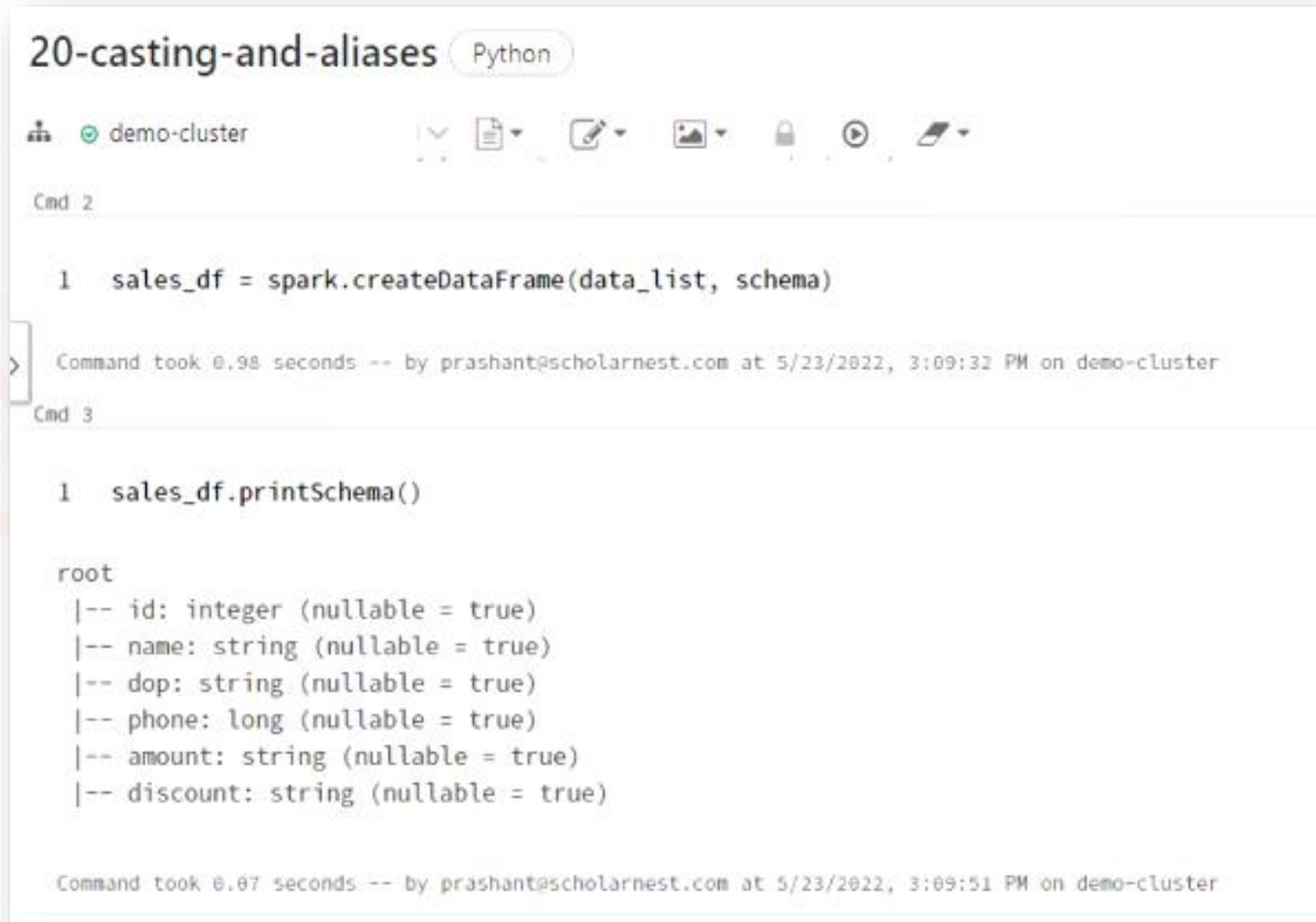
demo-cluster

Cmd 1

```
1 schema = 'id int, name string, dop string, phone long, amount string, discount string'
2
3 data_list = [(100, "Prashant", "2020-06-15", 9238614990, "12000", 18.5),
4              (101, "David", "2018-08-7", 8908617610, "15000", "nil"),
5              (102, "Simran", "14-05-2019", None, 3000000000, 21)]
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:08:17 PM on demo-cluster

Then we are creating a Dataframe and printing the schema of the Dataframe.



The screenshot shows a Jupyter Notebook window titled "20-casting-and-aliases" with a "Python" language selector. The interface includes a toolbar with icons for saving, running, and other actions. The notebook contains two code cells. The first cell, labeled "Cmd 2", contains the code `sales_df = spark.createDataFrame(data_list, schema)`. Below the code, the output shows the command execution time and user information. The second cell, labeled "Cmd 3", contains the code `sales_df.printSchema()`. The output shows the schema of the DataFrame, listing fields: id (integer), name (string), dop (string), phone (long), amount (string), and discount (string), all with nullable = true.

```
20-casting-and-aliases Python
```

demo-cluster

Cmd 2

```
1 sales_df = spark.createDataFrame(data_list, schema)
```

Command took 0.98 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:09:32 PM on demo-cluster

Cmd 3

```
1 sales_df.printSchema()
```

root

- id: integer (nullable = true)
- name: string (nullable = true)
- dop: string (nullable = true)
- phone: long (nullable = true)
- amount: string (nullable = true)
- discount: string (nullable = true)

Command took 0.07 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:09:51 PM on demo-cluster

Let's analyze our sample and list down some problems.

I already know some schema problems. Look at the schema below.

The id is an integer, the date of purchase is a string, the phone is a long, amount and discount are strings. That's not correct.

Cmd 3

```
1 sales_df.printSchema()
```

```
root
```

```
|-- id: integer (nullable = true)
|-- name: string (nullable = true)
|-- dop: string (nullable = true)
|-- phone: long (nullable = true)
|-- amount: string (nullable = true)
|-- discount: string (nullable = true)
```

```
Command took 0.07 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:09:51 PM on demo-cluster
```

I want to change the column names and data types to make them standard. Here is the new schema which I want to achieve. So I will be changing column names and some data types. For example, transaction_id is changed to string. Similarly, date_of_purchase is a date field. The purchase_amount is an integer, and the discount could be a double value. The best practice is to use long and avoid using integers. But I know it is purchase_amount, and it could not go bigger than two billion. If it is, that's a data problem for sure. I feel safe using an integer for purchase_amount, so let me keep it as an integer.

```
root
```

```
|-- transaction_id: string (nullable = true)
|-- customer_name: string (nullable = true)
|-- date_of_purchase: date (nullable = true)
|-- customer_phone: string (nullable = true)
|-- purchase_amount: integer (nullable = true)
|-- applied_discount: double (nullable = true)
```

Now let me analyze the data for new problems. Here is the sales_df Dataframe shown below. Eyeballing is one easy way to analyze and identify problems in a Dataframe.

Cmd 4

1 display(sales_df)

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	3000000000	21

Showing all 3 rows.

Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

So the id column is fine. It looks like integer values, and we want to convert it to string. That's a schema change, and I already listed it earlier.

Cmd 4

1

display(sales_df)

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	30000000000	21

Showing all 3 rows.

Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

The name column values are also acceptable, and we only want to change the column name to customer_name.

Cmd 4

1 display(sales_df)

>

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	3000000000	21

Showing all 3 rows.

Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

I can see the date in two formats. Some records are in YYYY-MM-DD, and I have a record in DD-MM-YYYY. That was a known problem, and I already knew it. However, one of the dates is 2018-08-7. The date component is a single digit. Ideally, it should be a double-digit number as 07. But it shows single digit 7. Let me note them down as new problems, and we will see how to fix them.

Cmd 4

1 display(sales_df)

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	30000000000	21

Showing all 3 rows.

Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

The phone number is also acceptable. It is null in some places, but that's not a data problem. The number is unknown, and null is the best value representing the missing number.

Cmd 4

1

display(sales_df)

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	30000000000	21

Showing all 3 rows.

Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

The amount looks good, with one exception. I assumed the purchase_amount to be a smaller number. But I can see a value larger than 2 billion. This record is incorrect for sure. So I have two options.

- 1. Fix the purchase_amount value
- 2. Separate this record into an error table.

I do not know how to fix the purchase_amount. I mean, I can set it to zero or null. But that will be wrong. I cannot assume any other value because we do not know the actual value. So it is best to separate this record into an error table. Even if I have nulls for the purchase_amount, those records should also go to the error table.

Cmd 4

1 display(sales_df)

>

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	3000000000	21

Showing all 3 rows.



Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

Now let's see the discount column. Some values are nil. That's a problem. I want to change these values to zero and convert the column to a double.

Cmd 4

1

display(sales_df)

(3) Spark Jobs

	id	name	dop	phone	amount	discount
1	100	Prashant	2020-06-15	9238614990	12000	18.5
2	101	David	2018-08-7	8908617610	15000	nil
3	102	Simran	14-05-2019	null	3000000000	21

Showing all 3 rows.

Command took 4.33 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:11:41 PM on demo-cluster

I used eyeballing to identify the problems. But that's not a recommended approach. I mean, you cannot identify data problems eyeballing a million records.

Data engineers use various techniques for analyzing and identifying data problems. However, statistical methods are the most commonly used approach. In this method, I am selecting the columns dop, amount, and discount. Then I am using describe() method to show some default statistical analysis of the given columns. The describe() is a Dataframe transformation.

It can perform fundamental statistical analysis on a Dataframe such as count, min, max, percentile, and mean values.

```
Cmd 5

1 sales_df.select("dop", "amount", "discount").describe().show()

> (2) Spark Jobs

+-----+-----+-----+-----+
|summary|      dop|      amount|      discount|
+-----+-----+-----+-----+
|  count|         3|           3|              3|
|   mean|       null|1.0000009E9|          19.75|
| stddev|       null|1.7320430133408928E9|1.7677669529663689|
|    min|14-05-2019|         12000|          18.5|
|    max|2020-06-15|30000000000|           nil|
+-----+-----+-----+-----+

Command took 4.40 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:14:01 PM on demo-cluster
```

You can see in the output below, the max value of the discount column is nil. And you know that's a problem. You were expecting decimal values in this column, but you have some strings.

Similarly, the mean and max values of the amount are too high. You were not expecting the amount to be in billions. So analysing your data and identifying data problems, outliers, unexpected values, etc., is a common activity for data engineers.

If you know your data, Spark methods, and their purpose, you can easily do the analysis.

```
Cmd 5

1 sales_df.select("dop", "amount", "discount").describe().show()

> (2) Spark Jobs

+-----+-----+-----+-----+
|summary|      dop|      amount|      discount|
+-----+-----+-----+-----+
|  count|         3|             3|             3|
|   mean|        null|1.0000009E9|         19.75|
| stddev|        null|1.7320430133408928E9|1.7677669529663689|
|    min|14-05-2019|         12000|         18.5|
|    max|2020-06-15|30000000000|          nil|
+-----+-----+-----+-----+

Command took 4.40 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:14:01 PM on demo-cluster
```

So we learned to identify problems, and I have the following list:

1. Convert id from integer to string and rename it as transaction_id.
2. Rename the name column to customer_name.
3. Convert the dop to date format and rename the column to date_of_purchase.
4. Rename the phone column to customer_phone
5. Convert the amount to an Integer value and filter out nulls and outlier values. Rename the column to purchase_amount
6. Convert amount to double, converting nil and null values to zero. rename the column to applied_discount

We have requirements to rename almost all the columns. And we also have a requirement to convert data types. You are also supposed to filter out some dirty records.

You can rename columns. I showed you two approaches:

1. `withColumnRenamed()`
2. `alias()`

If you rename multiple columns, we recommend using the alias and avoid `withColumnRenamed()`.

Why? Because each `withColumnRenamed()` adds a projection to your SQL plan and slows down your execution. So renaming multiple columns should be done using an alias in a single `select()` expression.

You already learned one approach to converting data types. We have two approaches to converting data types:

1. Using data type conversion functions such as `to_date()`, `string()`, `int()` etc.
2. Using `cast()` function

The id column is an integer. I want to convert it to a string. I told you about two approaches in the earlier slide, and here is the code using both approaches shown below.

You can use the `string()` built-in function to convert a column to a string. You can also use the `cast()` built-in function to convert a column to a string. The `string()` function is specific for converting a value to a string. However, the `cast` function is generic. You can use the `cast` function to convert a value to a string, integer, double, date, or whatever you want. The `cast` function supports all the Spark and Spark SQL datatypes.



```
20-casting-and-aliases Python
demo-cluster
Cmd 6

1 sales_df.selectExpr("string(id)",
2                       "cast(id as string)",
3                       "cast(amount as int)") \
4     .printSchema()

root
|-- id: string (nullable = true)
|-- id: string (nullable = true)
|-- amount: integer (nullable = true)

Command took 0.16 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:18:48 PM on demo-cluster
```

You can cast values as a different data type. The cast function will convert the value if it can. But if it cannot convert the value to a given data type, it will convert it to a null. You can see it here in the output below.

You can see the amount for 102 is now set to null.

We have a huge number that couldn't fit into an integer. So the casting method could not convert it to an integer, so this value becomes null. That's how all the data type conversion approaches work. You can try the case() function or try some other method to convert a data type. If the value is not good to be converted, it will become null.

Cmd 6

```
1 sales_df.selectExpr("string(id)",
2                       "cast(id as string)",
3                       "cast(amount as int)") \
4     .show()
```

► (3) Spark Jobs

id	id	amount
100	100	12000
101	101	15000
102	102	null



Command took 0.85 seconds -- by prashant@scholarnest.com at 5/23/2022, 3:19:06 PM on demo-cluster

Assignment:

You have the following requirements:

1. Convert id from integer to string and rename it as transaction_id.
2. Rename the name column to customer_name.
3. Convert the dop to date format and rename the column to date_of_purchase.
4. Rename the phone column to customer_phone and convert it to string
5. Convert the amount to an Integer value and filter out nulls and outlier values. Rename the column to purchase_amount
6. Convert discount to double, converting nil and null values to zero. Rename the column to applied_discount

You need the following tools to meet these requirements:

1. Rename columns
2. Convert Data types
3. nvl() function
4. Filter Records

You can use an alias to rename a column. You can use the cast() function to convert data types.

You can also use the to_date() function to convert a string date to a date type. You can use the nvl() function to check the null value and take an alternative value. You can use the where() method to filter unwanted records.

I leave you here so you can write code to meet the given requirements.
Here is the screenshot of the final result shown below. You can compare your solution output with the one shown here. The solution to this problem is given in the notebook. So you can download your study material and refer to the solution.

Expected Output

```
root
|-- transaction_id: string (nullable = true)
|-- customer_name: string (nullable = true)
|-- date_of_purchase: date (nullable = true)
|-- customer_phone: string (nullable = true)
|-- purchase_amount: integer (nullable = true)
|-- applied_discount: double (nullable = false)
```

transaction_id	customer_name	date_of_purchase	customer_phone	purchase_amount	applied_discount
100	Prashant	2020-06-15	9238614990	12000	18.5
101	David	2018-08-07	8908617610	15000	0.0



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com